

APPLIED MATH 3

EXTRA SUPPLIMENTAL

ANATOMY OF A CHARACTER IN UNREAL

PLAYER CHARACTERS IN UNREAL

- The concept of a Class Hierarchy is not extremely familiar to you, please review the supplementary notes on Class Hierarchies.
- The reason this is relevant to our discussion of Player Characters is because it is central to the architecture of the Unreal Engine.

PLAYER CHARACTERS IN UNREAL

- Something to consider that may be new if you are coming from other programming languages or game development tools is that Unreal is not a generic tool. It was created to make a specific game and so it brings much of the structure along with it.
- It is important to understand the differences between the different UE classes as they are each designed for a specific job.

IMPORTANT CLASSES TO KNOW: UObject

UObject

- This class is the base class of all classes we will talk about.
- It does not have a scene transform but is used at a low level by the engine such as providing garbage collection and virtual functions to support creating and using objects.
- If you just need a simple data class (and don't have a compelling reason to use raw C++ or have the object located in the scene, this is a good choice.
- API Documentation: <https://docs.unrealengine.com/5.0/en-US/API/Runtime/CoreUObject/UObject/UObject/>
- Overview in manual: <https://docs.unrealengine.com/5.0/en-US/unreal-objects/>

IMPORTANT CLASSES TO KNOW: AACTOR

AActor

- The base class that can be placed or spawned in a level.
- Extends UObject
- Can be subclassed for anything that needs to exist in a scene.
- Similar to a GameObject with Transform in Unity
- Best used for: Simple scene objects, anything that needs a location in the scene, or just needs to be placed somewhere in a scene (not attached to some other object).
- API Documentation: <https://docs.unrealengine.com/5.0/en-US/API/Runtime/Engine/GameFramework/AActor/>
- Unreal Manual: <https://docs.unrealengine.com/5.0/en-US/unreal-engine-actors/>

IMPORTANT CLASSES TO KNOW: APawn

APawn

- The base class that can listen for player or AI input.
- Extends AActor (note the A Prefix)
- Can be subclassed for player or NPC characters.
- Best used for: Player or NPC characters.
- API documentation: <https://docs.unrealengine.com/5.0/en-US/API/Runtime/Engine/GameFramework/APawn/>
- Overview in manual: <https://docs.unrealengine.com/5.0/en-US/pawns-in-unreal-engine/>

IMPORTANT CLASSES TO KNOW: APAWN

APawn

- We will be subclassing Pawn to create the Players for the assignments in this class.
- The concept of a Pawn in Unreal comes with some specific baggage.
- Pawns must be possessed by a Player, and typically each player controls one pawn at any given time.
 - If you are used to Unity, where every class listens for input events, this can be confusing at first.

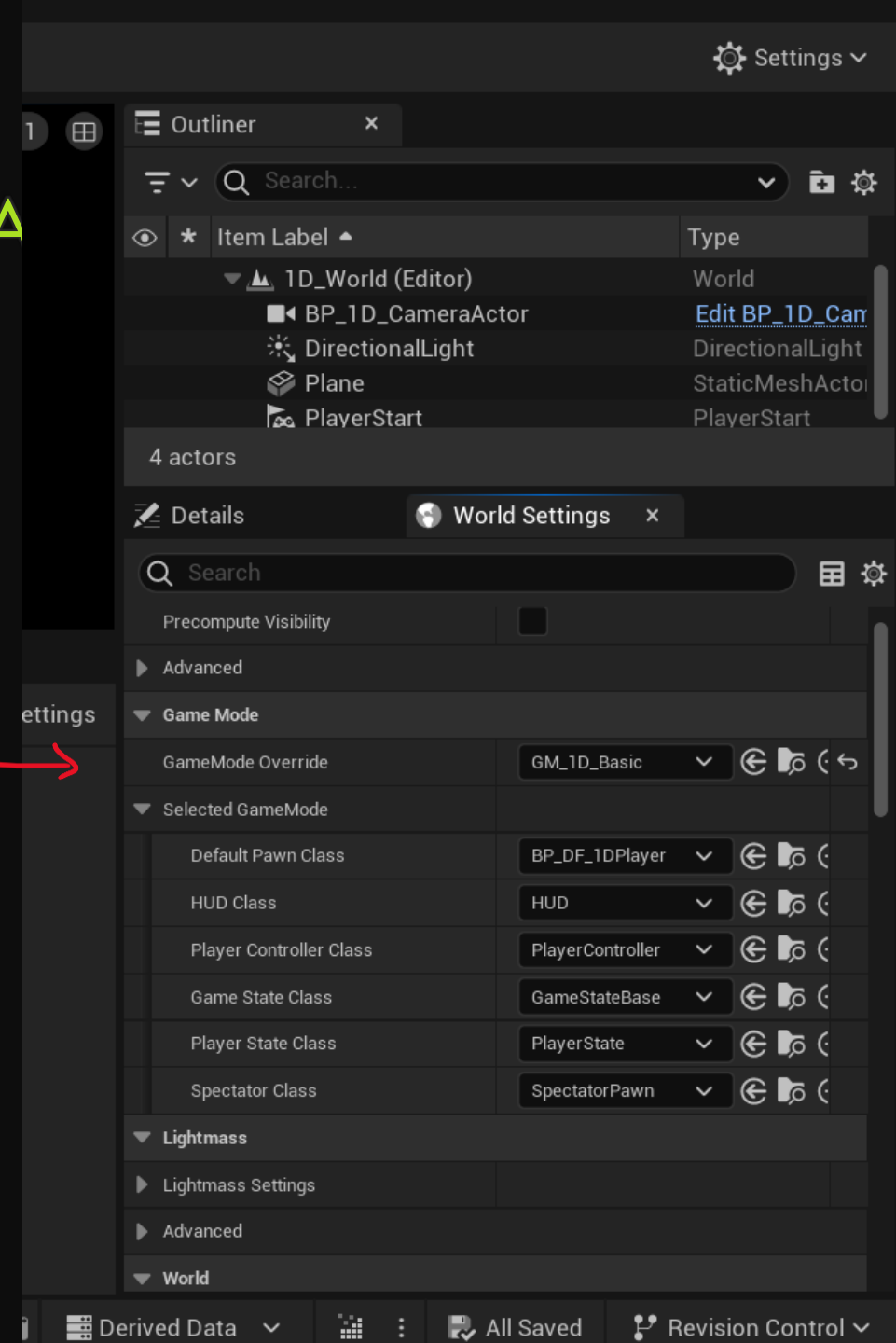
IMPORTANT CLASSES TO KNOW: APawn

APawn

- Pawns can be possessed in three ways
 1. Defined as the active Pawn in the GameMode or GameMode Override
 2. Activated by having the Auto Possess Player dropdown selected on the Blueprint Pawn itself.
(the object must exist in the scene at startup, and having no Pawn defined in the GameMode or GM Override)
 3. Activated in C++ or Blueprint code with the 'APlayerController::Possess()' function, which is used to possess a pawn programmatically.

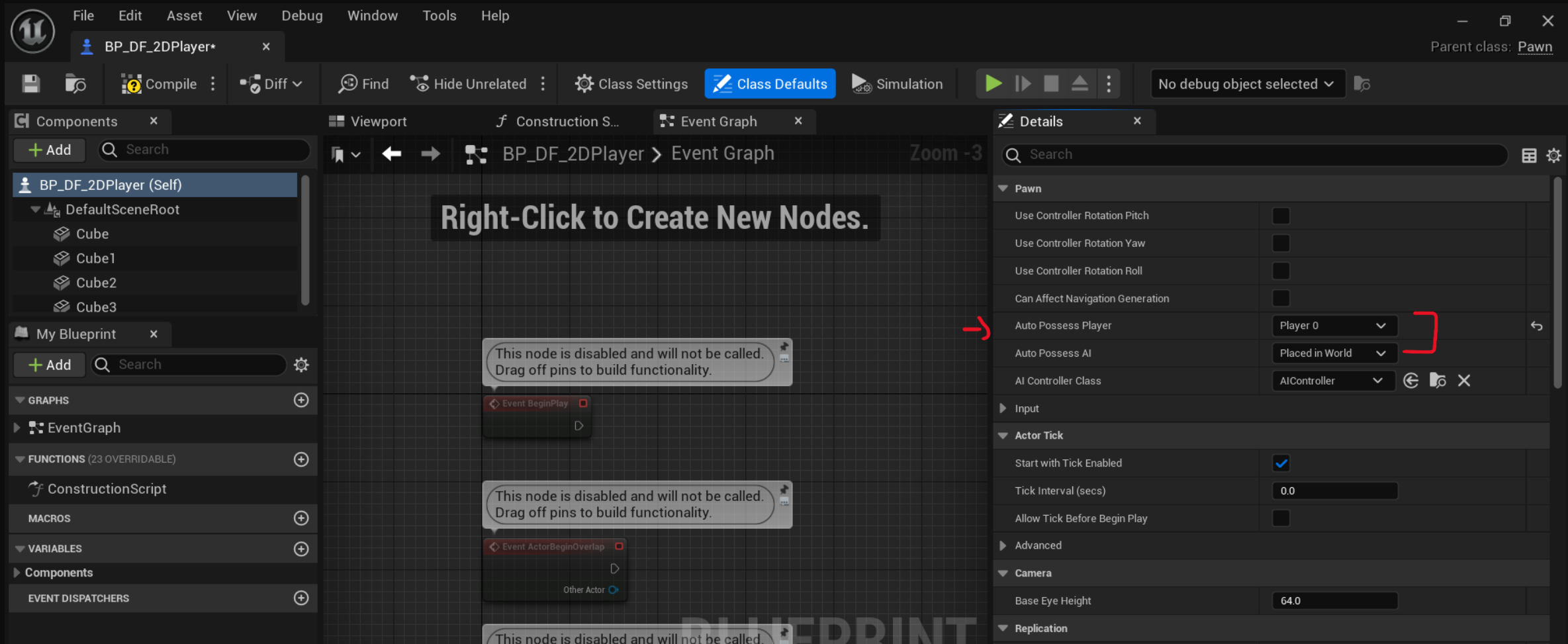
IMPORTANT CLASSES TO KNOW: A

Activating Pawn 1: Game Mode



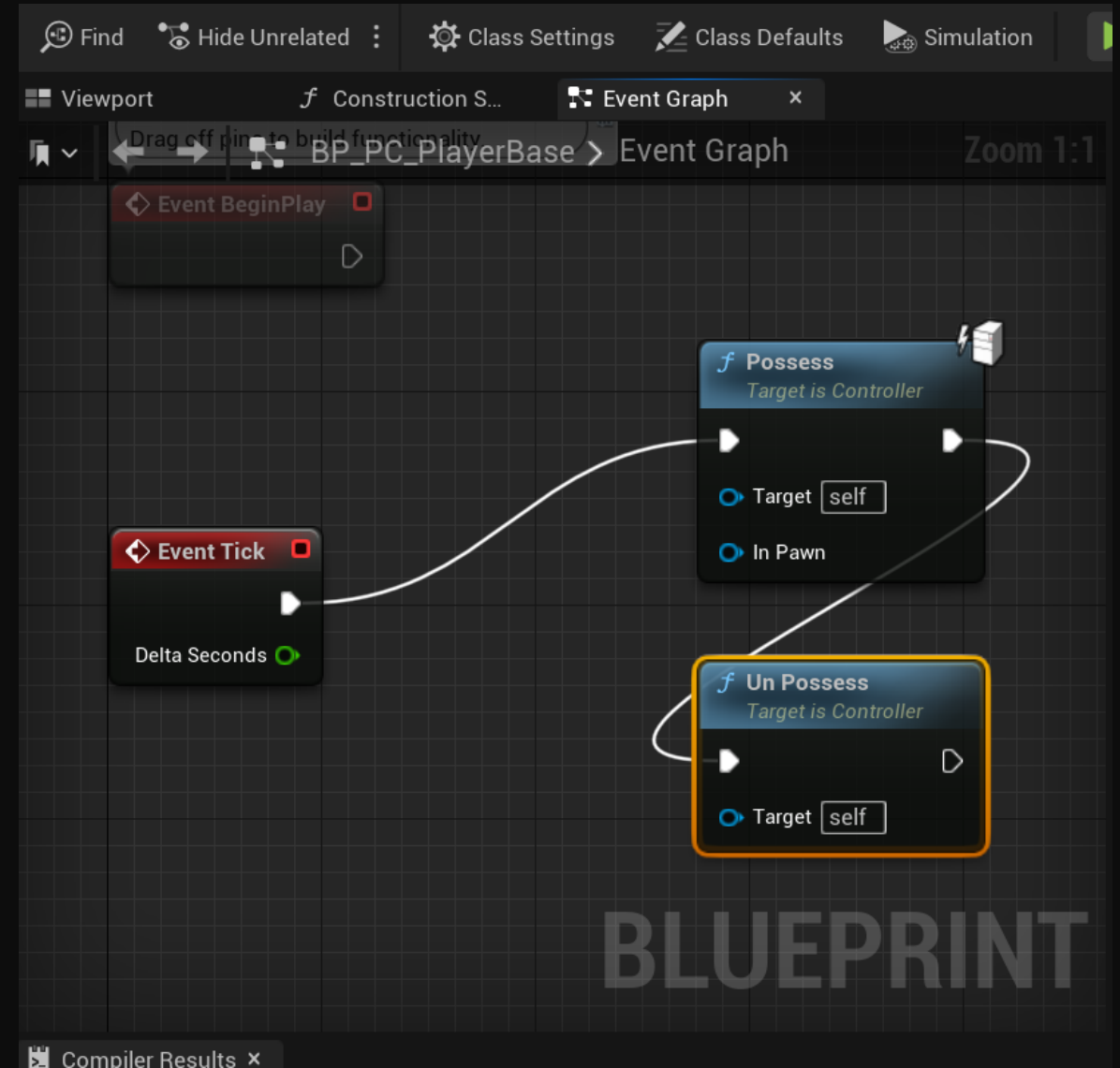
IMPORTANT CLASSES TO KNOW: APAWN

Activating Pawn 2: In Pawn Blueprint



IMPORTANT CLASSES TO KNOW: APAWN

Activating Pawn 3



IMPORTANT CLASSES TO KNOW: ADEFAULTPAWN

ADefaultPawn

- A Pawn subclass that comes with a spherical collision and built-in flying type movement functionality.
- This is a good option if you just need a very simple player you can navigate around the scene.
- API documentation <https://docs.unrealengine.com/5.0/en-US/API/Runtime/Engine/GameFramework/ADefaultPawn/>

IMPORTANT CLASSES TO KNOW: ACHARACTER

ACharacter

- A Pawn subclass that comes with a mesh, capsule collision and built-in humanoid movement functionality.
- This is usually considered the default for players and gives you a standard 1st or 3rd person character.
- It provides lots of functionality such as sliding on angled surfaces, jumping, duck/crawling and much more.
- While very powerful, it is not always the best option if we want full control over the player movement (or if we really want to learn about vectors!)
- We will NOT be using it in this class and will instead be implementing a similar movement logic ourselves.
- API documentation: <https://docs.unrealengine.com/5.0/en-US/API/Runtime/Engine/GameFramework/ACharacter/>

PAUSE HERE

- The concept of a Player has several parts in UE.
- The Pawn is considered the body of the player or NPC, but there is also the Controller, which controls a Pawn.
- Technically, you can just handle input directly in a Pawn (for very simple projects), however the better solution is to use the pawn in conjunction with a Controller.

CONTROLLERS

Controllers

- A controller is a non-physical Actor that can possess a Pawn (or Pawn subclass) and is used to control the Pawns actions.
- If the pawn is the body, this will be the brain.
- There is a 1-1 relationship between Controllers and Pawns by default, meaning each Controller will control one Pawn, and each Pawn will be controlled by a single controller.
- Manual: <https://docs.unrealengine.com/5.0/en-US/controllers-in-unreal-engine/>

CONTROLLERS

Controllers

The Controller class has two useful sub-classes.

1. PlayerController for human players
 2. AIController for AI players
- This means that by default, our Pawn architecture is already supporting being possessed by an AI as well (if we structure it correctly).
 - This structure also make it very easy to have a player switch Pawns in game (such as entering or exiting a vehicle) since control and body are already separate concepts.
 - It is also important for games where you might have multiple lives (such as a deathmatch) or if you have multiple players in a single client.
 - The Controller also persists throughout the game, where the Pawn can be destroyed and respawned. This makes the controller a useful place for information that belongs to the player but outlives the Pawn, such as a score.

CONTROLLERS

Controllers

- The Camera also exists as part of the Controller, and although the Controller doesn't have a position of its own, it does have a rotation.
- This was one of the single most confusing things I found in UE and I have struggled to find a good answer to why it is organized this way, other than it just is.

COMPONENTS

Components

- (Why does everything start with C? So confusing...)
- A Component is a class that can be attached to an Actor.
- It doesn't have its own position in the scene, but instead exists as part of an Actor in the scene.
- This is a useful place for modular functionality (an Inventory, Health) that can be then reused on many different pawns.

ADD USEFUL METHODS HERE

- Possess/Unpossess
- Spawning Actors/Pawns
- Basic movement.
- What else?